IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.Doi Number

Convolutional Neural Networks-Based Locating Relevant Buggy Code Files for Bug Reports Affected by Data Imbalance

GUANGLIANG LIU[1,2], YANG LU[1,3], KE SHI[1], JINGFEI CHANG[1], and XING WEI[1,3]

[1] School of Computer Science and Information Engineering, Hefei University of Technology, Hefei, 230009, China

[2] School of Software, Nanyang Normal University, Nanyang, 473061, China

[3] Engineering Research Center of Safety Critical Industrial Measurement and Control Technology of the Ministry of Education, Hefei, 230009, China

Corresponding author: Yang Lu (luyang.hf@126.com).

This work was supported in part by the National Key Research and Development Program under Grant 2018YFC0604404 and Grant 2016YFC0801804, in part by the National Natural Science Foundation of China under Grant 61806067, and in part by the Fundamental Research Funds for the Central Universities under Grant PA2019GDPK0079.

ABSTRACT Software bug localization is very important in software engineering, but it is also complicated and time consuming. To improve the efficiency of developers, researchers have developed various traditional bug localization and machine learning bug localization methods. In this paper, we propose a novel method that improves bug localization performance. First, surface lexical correlation matching between bug reports and source code files is used to obtain features by deep neural network. Second, to solve the lexical gap between bug reports and source code files, semantic correlation matching between them is used to obtain features based on word embedding and sentence embedding by deep neural network. Then, the joint features obtained by the surface lexical and semantic correlation matching are fused into a unified feature representation for bug reports and source code files. In addition, since our experimental datasets are imbalanced data, we use a focal loss function to solve the impact of data imbalance. Finally, our method obtains the relatively high bug localization performance compared to other classic methods.

INDEX TERMS Convolutional neural network, bug localization, surface lexical correlation matching, semantic correlation matching, bug report, focal loss.

I. INTRODUCTION

Software defect fixes in the software lifecycle have always been very important. General software defects are fed back to the development team by bug reports, and the development team fixes the bugs based on the reports. The key to addressing bug reports is to locate relevant source code files that contain defects. There are many methods for bug localization, such as traditional bug localization and machine learning bug localization methods.

Some researchers use traditional methods for bug localization. Lukins et al. [1] introduced an approach to measure the similarity between bug reports and source code files based on Latent Dirichlet Allocation (LDA) for bug localization. Gay et al. [2] calculated similarities between bug reports and source code files by Vector Space Model (VSM) for bug localization. Zhou et al. [3] proposed the famous BugLocator method, which uses a revised VSM to rank all source code files according to the text similarity, taking into account similar bugs previously fixed. Saha et al. [4] used the structure information of source code files to improve the bug localization performance. Klaus et al. [5] used text, stack trace, comment, structure and change history of source code files to locate buggy code files at the file and method level. Wong et al. [6] presented the BRTracer method, which improves the accuracy of bug localization by segments and stack-trace information. Wang et al. [7] proposed the AmaLgam+ method, which uses similar report, structure, and other information to locate relevant buggy files. These traditional methods use feature attributes of bug reports and source code files for bug localization. These methods generally only consider surface lexical correlation matching of bug reports and source code files, ignoring semantic correlation matching, and because of the lexical gap, the performance is not high for bug localization. In addition, there are many traditional bug localization methods that require other relevant information

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

that needs to be collected from software repositories. Moreover, some information is not easy to be collected, and, this also increases the complexity of some models.

Machine learning methods in bug localization generally include learning-to-rank and deep learning. Ye et al. [8] used the learning-to-rank algorithm to locate relevant buggy code files. Later, Ye et al. [9] used some related techniques such as word embedding to improve the similarity between bug reports and source code files. However, although this method uses word embedding to improve bug localization performance, it needs to collect additional relevant documents to train word embedding. In addition, the accuracy improvement is also limited. Recently, some researchers have used deep learning methods for bug localization. Lam et al. [10] used deep learning to locate relevant buggy code files. The method also considers the bug-fixing history for improving the bug localization performance. Huo et al. [11] proposed the NP-CNN method, which extracts the features of bug reports and source code files by convolutional neural networks, and then fuses their features to obtain the results of bug localization. Huo et al. [12] proposed the LS-CNN method, which fuses the extracted features by long short term memory networks and convolutional neural networks to obtain the results of bug localization. Xiao et al. [13] presented the DeepLoc method, which obtains vector of bug reports and source code files by word2vec and sent2vec to locate relevant buggy files by convolutional neural networks.

In these bug localization methods based on deep learning, some consider surface lexical correlation matching between bug reports and source code files, regardless of their semantic correlation matching. However, due to the lexical gap between bug reports and source code files, the bug localization performance is not high. Instead, some only consider semantic correlation matching, regardless of surface lexical correlation matching. The bug localization performance is also not high. Therefore, to more accurately locate relevant buggy code files for bug reports, we consider not only the surface lexical correlation matching between bug reports and source code files but also the semantic correlation matching based on word embedding and sentence embedding. Word embedding has proven to be an important technology in a variety of natural language processing (NLP) tasks. [14, 15]. In addition, since each bug report is only related to a small number of source code files, it causes data imbalance for bug localization, and focal loss is used to solve the data imbalance. A part-of-speech (POS) tagger and abstract syntax trees (ASTs) are also used in bug reports and source code files, respectively, to determine the weights of corresponding words to calculate semantic correlations matching.

The main contributions of our work are as follows:

1. We propose a novel method called joint surface lexical and semantic correlation matching based on convolutional neural networks (SLS-CNN) for bug localization.

2. We use the loss function, named focal loss, to solve the imbalance data for bug localization.

3. We use a POS tagger and ASTs to determine the weights of corresponding words to calculate semantic correlation matching between bug reports and source code files.

The remainder of this paper is organized as follows. Section II reviews the background related to SLS-CNN. Section III gives a description of our approach. Our experimental preparations and results are presented in Section IV. Threats to validity are reviewed in Section V. Finally, we conclude this paper in Section VI.

II. BACKGROUND

In this section, convolutional neural networks, word2vec and doc2vec, imbalanced learning in software engineering and focal loss are presented.

A. CONVOLUTIONAL NEURAL NETWORKS

Deep neural networks are usually more difficult to train than traditional machine learning, but deep neural networks can acquire more powerful capabilities than traditional machine learning. Convolutional neural networks are very important in deep learning-related neural networks. In the 1960s, Hubel and Wiesel [16] discovered the unique network structure of the local sensitivities and directional selection of neurons in a cat's visual cortex, which can effectively improve the training performance of neural networks. In the 1980s, Fukushima [17] presented a hierarchical architecture, which is very similar to convolutional neural networks, but there is no back propagation mechanism. Later, more researchers entered the field to improve network performance.

Convolutional neural networks are mainly used in image processing. In recent years, convolutional neural networks have been gradually used in natural language processing and programming language processing, and have achieved good results [18]. Convolutional neural networks typically contain multiple layers. In addition to the common input and output layers, it also contains a convolutional layer, a pooling layer, and a fully connected layer. The input layer inputs data as an image pixel, text, or program code. The main function of the convolutional layer is to obtain local features of input data. The feature map of the previous layer is convoluted with the convolution kernel, and the result of the convolution is output by the activation function to form the neurons of the layer, thereby forming the feature map of the layer, named the feature extraction layer. The function of the pooling layer is to reduce the dimensions of the features obtained by the convolutional layer. The pooling layer splits the input data into nonoverlapping regions and reduces the spatial resolution of the network by pooling operations for each region. For example, the maximum pooling gets the maximum value in the selection area, and the average pooling gets the average value in the selection area. This operation eliminates the offset and distortion of the data. The fully connected layer is applied after the input data have undergone several convolutions and pooling operations. The

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

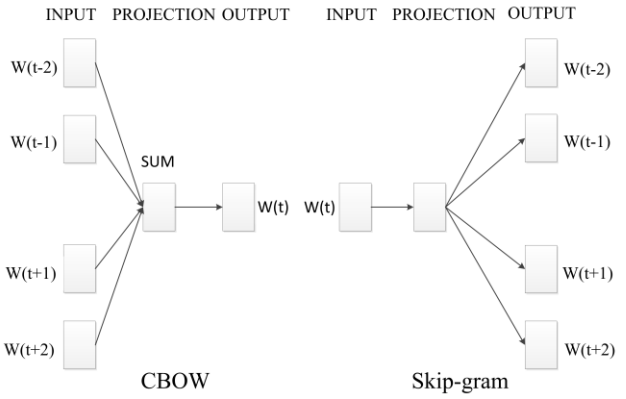

FIGURE 1. CBOW and Skip-gram methods.

output is a set of data that is fully connected. Finally, multiple sets of data are combined into one set of data. The output layer performs operations based on requirements (classification or regression, etc.).

Convolutional neural networks include forward propagation and back propagation.

Forward propagation includes related mathematical formulas. The output of the convolutional layer is computed in (1). The output of the pooling layer is computed in (2). The output of the fully connected layer is computed in (3). The output of the output layer is computed in (4).

$$a^l = ReLU\left(a^{l-1} * W^l + b^l\right) \quad (1)$$

$$a^l = pool\left(a^{l-1}\right) \quad (2)$$

$$a^l = \sigma\left(W^l a^{l-1} + b^l\right) \quad (3)$$

$$a^l = softmax\left(W^l a^{l-1} + b^l\right) \quad (4)$$

where a^l is the output of each layer. W and b refer to the parameters of hidden layer. $ReLU$, $pool$, σ, and $softmax$ are functions, and l refers to the sequence number of layers for deep learning.

Back propagation includes related mathematical formulas (using the batch gradient descent method as an example to describe the back propagation algorithm). The output of the fully connected layer is computed in (5). The output of the convolutional layer is computed in (6). The output of the pooling layer is computed in (7).

$$\delta^{i,l} = \left(W^{l+1}\right)^T \delta^{i,l+1} \odot \sigma'\left(z^{i,l}\right) \quad (5)$$

$$\delta^{i,l} = \delta^{i,l+1} * rot180\left(W^{l+1}\right) \odot \sigma'\left(z^{i,l}\right) \quad (6)$$

$$\delta^{i,l} = upsample\left(\delta^{i,l+1}\right) \odot \sigma'\left(z^{i,l}\right) \quad (7)$$

where $\delta^{i,l}$ refers to the output of each layer. $rot180(\ )$ is the convolution kernel that was rotated 180 degrees. $upsample(\ )$ completes the logic of the pooling error matrix amplification and error redistribution. $z^{i,l}$ is the tensor, and l is the sequence number of layers for deep learning.

B. WORD2VEC AND DOC2VEC

Word embedding is a densely distributed representation of words. Each word vector usually has tens to hundreds of dimensions [19], which is in stark contrast to the thousands of dimensions required for word sparse representation. For example, one-hot encoding is an example of sparse representation. With the development in deep neural network, word embedding has been effectively trained in many ways, which was widely used in a variety of NLP tasks [20, 21]. Ever due to the in-depth study of the neural probabilistic language model by Bengio et al. [22, 23], the distributed vector models based on neural network have been widely developed. A word vector is a real-valued vector representation of fixed-size vocabulary based on learning a large number of text corpora, which allows words with similar semantics to have similar representations. There are a variety of methods for constructing word embeddings, such as the word2vec model [20, 24] and the GloVe model [25]. Fig. 1 shows that word2vec has two training methods, namely, CBOW and skip-gram. Word2vec has two methods to speed up the training of words. One is hierarchical softmax, and the other is negative sampling [26, 27]. Compared to hierarchical softmax, negative sampling abandons the Huffman tree structure from the projection layer to the output layer and changes to a full connection. The word vector trained by word2vec works well, and it can measure the similarity between different words.

The skip-gram optimization objective is computed, as shown in (8).

$$L(\theta) = \prod_j^S \left(\prod_{i_j=1}^{T_j} \left(\prod_{-c_{ij} < u_{ij} < c_{ij},\, j \neq 0} p\left(w_{u_{ij}+i_j} \mid w_{i_j}\right) \right) \right) \quad (8)$$

The corpus is a sequence of sentences consisting of S sentences (order is not important), and the entire corpus has V words to construct a likelihood function. T_j is the number of words in the j-th sentence. $w_{u_{ij}+i_j}$ is one of the c_{ij} words around the word w_{i_j}.

The CBOW optimization objective is computed, as shown in (9).

$$L(\theta) = \prod_j^S \left(\prod_{i_j=1}^{T_j} p\left(w_{i_j} \mid Context_{i_j}\right) \right) \quad (9)$$

The corpus is a sequence of sentences consisting of S sentences (order is not important), and the entire corpus has V words to construct a likelihood function. T_j is the number of words in the j-th sentence. w_{i_j} is a predicted word. $Context_{i_j}$ refers to the context of a sentence.

The previous section describes how a word forms a unique vector representation through the word2vec model. However, there are also many methods for constructing a sentence vector, and doc2vec is one of them. Doc2vec, also called a paragraph vector, was proposed by Tomas Mikolov based on the word2vec model [20]. It has some advantages; for example, it can accept sentences of different lengths as training samples, and predict a vector to represent different sentences. The structure of the model potentially overcomes the shortcomings of bag-of-words that ignore word order and syntax. Le et al. [28] proposed two models for learning

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

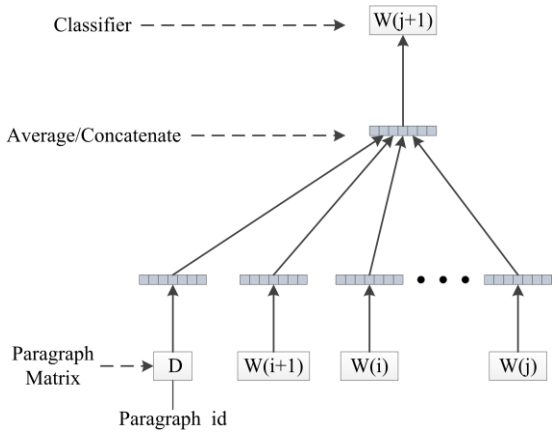

FIGURE 2. PV-DM method.

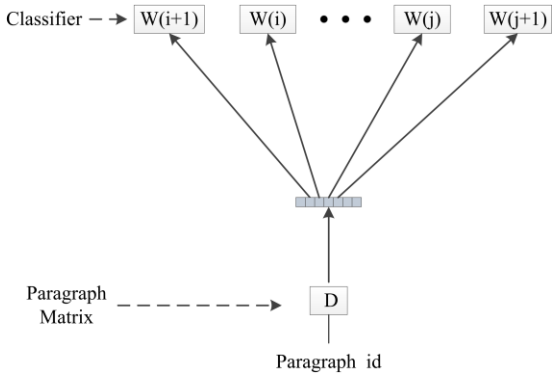

FIGURE 3. PV-DBOW method.

sentence and document distributed representations (paragraph vector). The Distributed Memory version of Paragraph Vectors (PV-DM) was the first model to learn paragraph vectors, similar to the CBOW model in word2vec, as shown in Fig. 2. The distributed Bag-of-Words version of Paragraph Vector (PV-DBOW) was the second model to learn paragraph vectors, similar to the skip-gram model in word2vec, as shown in Fig. 3. The advantage of PV-DM and PV-DBOW is that they can acquire not only sentence vectors but also document vectors. Calling doc2vec is quick and easy, and the implementation is integrated into the gensim package (https://radimrehurek.com/gensim/).

C. IMBALANCED LEARNING IN SOFTWARE ENGINEERING

In many cases, data sometimes has an imbalance problem. That is, one class of data contains considerably more data than other classes. The methods for solving data imbalance are usually divided into two categories: through sampling and through cost-sensitive methods. The sampling methods mainly change the distribution of original data, through oversampling, undersampling and synthetic sampling. The cost-sensitive methods use different categories of samples to be misclassified to generate different costs to solve the data imbalance problem. In addition, many studies show that there is a strong link between cost sensitivity and data imbalance, and using cost-sensitive methods to solve the imbalance problem is better than using the sampling methods.

Related technologies for data imbalance are widely used in software engineering. Wang et al. [29] improved the defect prediction performance based on the class imbalance. The dynamic AdaBoost.NC method was proposed to automatically adjust parameters during the training process without any parameters being defined in advance. This method is more efficient than the original AdaBoost.NC. Arar et al. [30] proposed a hybrid classifier that combines an artificial neural network (ANN) with an artificial bee colony (ABC). ABC is used to optimize the parameters of the ANN, which adds cost sensitivity to the ANN by an error function. Sheng et al. [31] developed the STEP system, which obtained maximum net profit, and converted the maximum net profit problem into a cost-sensitive problem. Xie et al. [32] proposed a cost-sensitive margin distribution optimization loss function and applied it to a deep neural network, which can handle the imbalance of software defect datasets. Huo et al. [11] proposed an unequal misclassification cost according to the imbalance ratio and trained a fully connected network in a cost-sensitive manner. Huo et al. [12] randomly dropped some negative samples to counteract the negative influence of data imbalance.

To improve the bug localization performance and handle the imbalanced data, we introduce a focal loss function and apply it to convolutional neural networks.

D. FOCAL LOSS

Lin et al. [33] first proposed focal loss, which was used for object detection. Focal loss mainly solves the problem of serious imbalances between positive and negative samples. It can be applied in many other research areas. For example, for bug localization, there are few source code files related to bug reports, and many source code files unrelated to bug reports, and for fraud detection, normal events outnumber abnormal events. These are imbalanced datasets. If you use common methods to solve these problems, it is difficult to meet the accuracy requirements. The focal loss function reduces the weights of a large number of simple negative samples during training. The cross-entropy loss function is computed in (10). Focal loss is formed based on the cross-entropy loss function.

$$L = -\left(y \log \tilde{y} + (1-y)\log(1-\tilde{y})\right) = \begin{cases} -\log \tilde{y}, & \text{y=1} \\ -\log(1-\tilde{y}), & \text{y=0} \end{cases} \quad (10)$$

where $\tilde{y}$ is the output of the activation function, with values between 0 and 1. It can be seen that the cross-entropy loss is smaller for the positive sample with a larger output probability. For negative samples, the smaller the output probability is, the smaller the loss. The loss function is slow in the iterative process of a large number of simple samples, and the function may not be optimized. The focal loss function is computed in (11).

$$L_{fl} = \begin{cases} -(1-\tilde{y})^{\gamma} \log \tilde{y}, & \text{y=1} \\ -\tilde{y}^{\gamma} \log(1-\tilde{y}), & \text{y=0} \end{cases} \quad (11)$$

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

where γ >0 makes it possible to reduce the loss of easily classified samples and more attention to difficult misclassified samples. γ adjusts the rate at which simple sample weights are reduced. When γ is 0, it is the cross-entropy loss function. The degree of influence of the adjustment factor increases as γ increases.

For example, γ is 2, and for a positive sample, the prediction result of 0.95 is definitely a simple sample, so the γ power of (1-0.95) is small, and the loss function value will decrease. The sample with a predicted probability of 0.3 has a relatively large loss. For negative samples, the predicted probability of 0.1 should be much smaller than the predicted probability of 0.7. For a predicted probability of 0.5, the loss is only reduced by 0.25 times, so more focus is on the indistinguishable samples. This reduces the impact of simple samples, and the impact of a large number of samples with a low probability of prediction can be more effective. In addition, the balance factor α is added to balance the proportion of positive and negative samples in (12).

$$L_{fl} = \begin{cases} -\alpha(1-\tilde{y})^{\gamma}\log\tilde{y}, & \text{y=1} \\ -(1-\alpha)\tilde{y}^{\gamma}\log(1-\tilde{y}), & \text{y=0} \end{cases} \quad (12)$$

III. OUR APPROACH

Fig. 4 shows the overall framework of our model. The whole architecture consists of three modules.

1. In module 1, the surface lexical correlation matching module utilizes convolutional neural networks to operate the interaction matrix of bug reports and source code files.

2. In module 2, the semantic correlation matching module is a hierarchical multistage structure, and the features of bug reports and source code files are extracted through convolutional neural networks.

3. In module 3, the feature connection optimization module links surface lexical and semantic correlation features and then optimizes the model based on focal loss.

To improve the speed of training, we use the cosine similarity between any given bug report and the source code files to select the top 300 irrelevant source code files for training. In the paper, we obtain the number (n_{sbd}) of sentences from the description of each bug report, the maximum number (n_{sbs}) of words from the summary of each bug report, the number (n_{sc}) of lines from each source code file, the number (n_{bsb}) of words from the bug report and the number (n_{bsc}) of words from each source code file. For the training efficiency of our model, n_{sbs} is the maximum value, and n_{sbd} , n_{sc} , n_{bsb} and n_{bsc} are not the maximum values. Statistics show that if the n_{sbd} , n_{sc} , n_{bsb} and n_{bsc} are set too small, the bug report and source code file cannot be fully expressed, and if the setting is too large, space is wasted. If the length of the bug report and source code file is less than the defined target length, they are padded by zeros. At the same time, Ye et al. [9] confirmed that the word vectors obtained by training the Wiki corpus and the project-specific corpus with word2vec have similar performance, and thus,

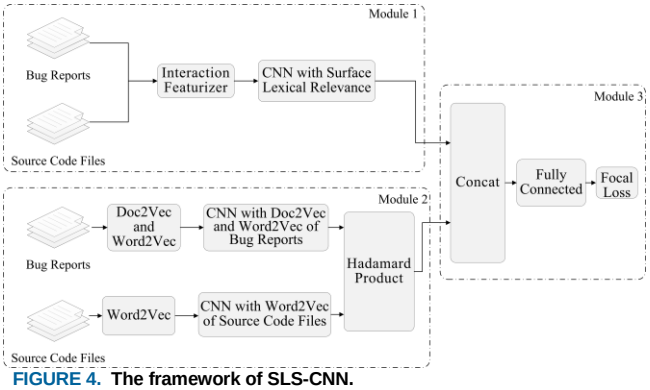

FIGURE 4. The framework of SLS-CNN.

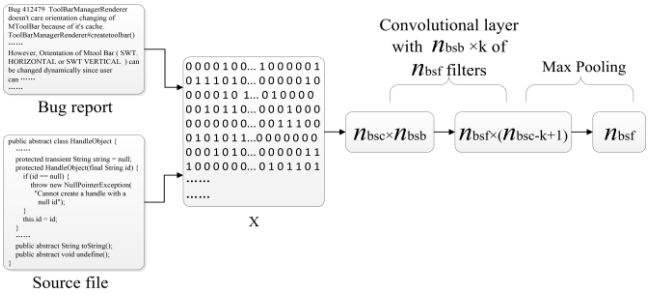

FIGURE 5. Extracting features from the interaction matrix of bug reports and source code files.

the word vectors used in our method are obtained by training the Wiki corpus with word2vec.

A. MODULE 1 –SURFACE LEXICAL CORRELATION MATCHING

Surface lexical correlation matching is used for bug localization by the estimated correlation between bug reports and source code files based on the number and position of words.

Preprocessing of bug reports and source code files is important for surface lexical correlation matching. First, we split compound words into individual words for bug reports and source code files. For example, BcelClassWeaver is split into three words Bcel, Class and Weaver based on the capital letters. Then we remove punctuation, numbers and stop words, and convert uppercase to lowercase letters, etc. from the bug report. Then, in addition to removing Java language keywords for source code file, its preprocessing is the same as bug report preprocessing. In addition, in this paper, stemming of bug reports and source code file uses Porter stemming (https://tartarus.org/martin/PorterStemmer/).

In this module, we use a bug report and a source code file to generate an $n_{bsc} \times n_{bsb}$ interaction matrix X, which is the key to surface lexical correlation matching. The matrix is obtained whether the relevant words of the bug report appear in the source code file. Generating the interaction matrix is very simple. It only needs to compare each word r_i in the bug report with each word s_j in the source code file. If they are equal, the corresponding position of the interaction matrix is 1. If they are not equal, the corresponding position of the interaction matrix is 0. The final result is an interaction

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

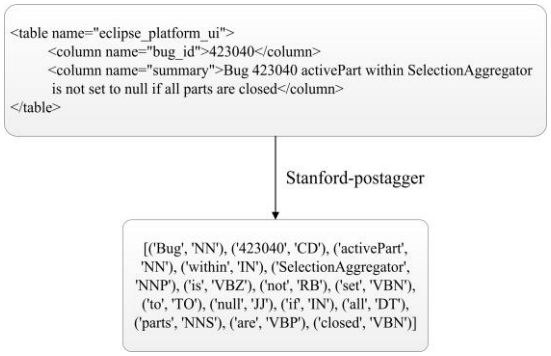

FIGURE 6. The POS tagging results of the Eclipse UI for bugID= "423040".

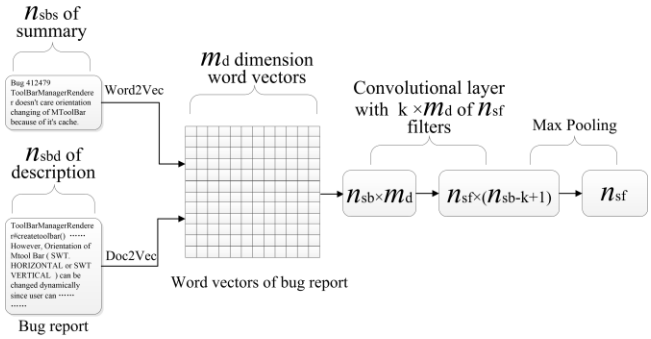

FIGURE 7. Extracting features from bug reports.

matrix containing only elements 0 and 1. Although the interaction matrix captures the surface lexical correlation matching of source code files and bug reports well, it does not retain semantic information about the words. Therefore, surface lexical correlation cannot obtain the specific information of words from the corpus, nor can it obtain the correlation information between different words. As shown in Fig. 5, in the interaction matrix, the vertical is the vocabulary of the source code file, and the horizontal is the vocabulary of the bug report. The interaction matrix is first entered into the convolutional layer with n_{bsf} convolution kernels, each convolution kernel having a size of $n_{bsb} \times k$ and a step of 1. The output of the convolutional layer is entered into the pooling layer, and n_{bsf} is obtained as the representation of surface lexical correlation features.

B. MODULE 2 –SEMANTIC CORRELATION MATCHING

Because surface lexical correlation has certain deficiencies in bug localization, semantic correlation can compensate for the lack of surface lexical correlation to some extent. The semantic correlation of bug reports and source code files can be obtained based on word embedding and sentence embedding by deep neural networks. The preprocessing of bug reports and source code files is the same as the preprocessing of module 1.

Semantic correlation matching first uses word2vec and doc2vec to obtain the low-density vector representation of words and sentences. The word vectors and sentence vectors (m_d dimension) can reflect the semantic information of words and sentences, and then calculate semantic correlation matching between bug reports and source code files based on word vectors and sentence vectors. In this paper, the vector matrices for bug report and source code file are represented as $n_{sb} \times m_d$ and $n_{sc} \times m_d$, respectively.

In addition, in each bug report, the description is not as important as the summary [34]. Regardless of summary or description, they are written in natural language. The description of the bug report forms sentence vector by doc2vec, and the vector matrix for the description is $n_{sbd} \times m_d$. Doc2vec has difficulty retaining important information for summary because the summary is always written containing useful information; therefore, the summary of the bug report cannot directly form a sentence vector by doc2vec. We convert the summary into a word vector by word2vec, and the vector matrix for the summary is $n_{sbs} \times m_d$.

In general, noun-based sentence terms are more important than other POS in bug reports. Therefore, POS tagging techniques are used to label the summary field of each bug report [35, 36]. In this paper, we use the POS tagger, named Stanford tagger (http://nlp.stanford.edu/software/tagger.shtml), to mark the POS of the summary field, and extract the nouns in the summary to increase their corresponding weights. For example, in Fig. 6, the top part is the summary part, which belongs to the number 423040 Eclipse UI bug report, and the bottom part shows the POS tagging results of the summary by Stanford tagger, where 'Bug', 'activePart', 'SelectionAggregator' and 'parts' are nouns. Because 'SelectionAggregator' can directly specifies the buggy source code file for 'SelectionAggregator.java', this also proves the importance of nouns. Moreover, Tian et al. [37] confirmed that the Stanford tagger and TreeTagger achieved the highest accuracy on sample bug reports compared to seven other POS taggers.

In this case, Fig. 7 shows the process by which bug reports are extracted features through the convolutional neural network. n_{sbd} and n_{sbs} are combined into n_{sb} for the maximum number for the bug report, and then, we use a convolutional layer with $k \times m_d$ of n_{sf} filters to form a $n_{sf} \times (n_{sb} - k + 1)$ matrix. Through the max pooling layer processing, the n_{sf}-dimensional vector is formed as features of the bug report extracted by the convolutional neural network.

In bug localization, some tokens in the source code file are very important, such as method and class name. Because source code file is structural, we rely on ASTs to obtain method and class names, and increase their corresponding weights for calculating semantic correlation. In this paper, we use javalang (https:// github.com/c2nes/javalang) to parse source code files into ASTs. In the current javalang version, it is designed using the Python language. Javalang provides a lexer and parser for java based on the Java language spec. In Fig. 8, we use javalang to extract the method and class names of Eclipse UI's HandleObject.java.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

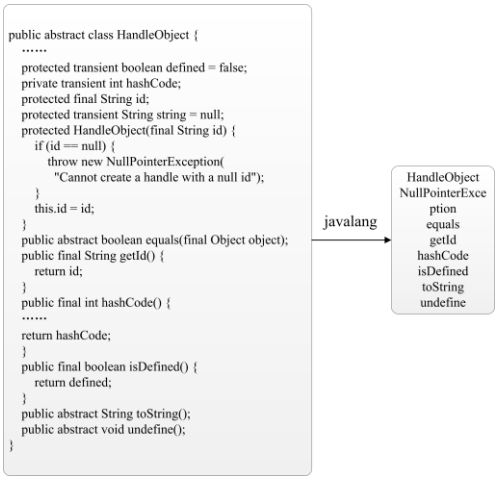

FIGURE 8. The javalang results of the Eclipse UI for HandleObject.java.

Fig. 9 shows the process by which source code files are extracted features through a convolutional neural network. First, each line for the source code files is converted into a word vector by word2vec. The dimension of the word vector is m_d, and the vector matrix for the source code file is $n_{sc} \times m_d$. Then, we use a convolutional layer with $k \times m_d$ of n_{sf} filters to form an $n_{sf} \times (n_{sc} - k + 1)$ matrix. Through the max pooling layer processing, the n_{sf}-dimensional vector is formed as features of the source code file extracted by the convolutional neural network.

The n_{sf}-dimensional vector can be obtained as features for the bug report. Accordingly, the n_{sf}-dimensional vector can be obtained as features for the source code file. To match the bug report and the source code file, this paper performs the Hadamard product between the obtained source code file features and bug report features in (13).

$$HP = n_{sf} \circ n_{sf} \tag{13}$$

HP is the semantic correlation features of source code file and bug report.

C. MODULE 3 –FEATURES FUSION LAYER

In this section, the features obtained by module1 and module2 are fused into a unified feature representation for bug localization. In addition, there are few source code files related to bug reports, and many source code files unrelated to bug reports. The imbalance between the bug reports and source code files increases the difficulty in obtaining the high bug localization performance based on extracted features. To improve the bug localization performance, inspired by [33], we train our model to learn the required parameters by minimizing the cost function in (14).

$$L_{fl} = \frac{1}{N \times K} \sum_{i=1}^{N} \sum_{k=1}^{K} \left(-\alpha y_i^k \left(1 - \tilde{y}_i^k\right)^{\gamma} \log \tilde{y}_i^k - (1-\alpha)\left(1 - y_i^k\right)\left(\tilde{y}_i^k\right)^{\gamma} \log\left(1 - \tilde{y}_i^k\right)\right) \tag{14}$$

where *N* refers to the number of bug reports, and *K* refers to

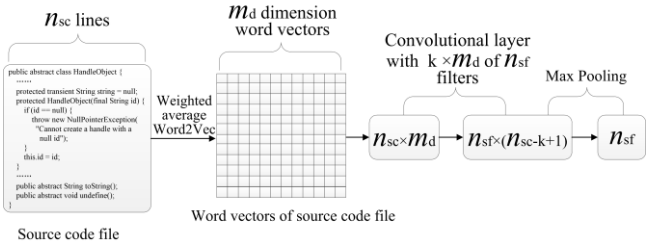

FIGURE 9. Extracting features from source code files.

the number of source code files. y_i^k refers to the label of whether the *i*-th bug report and the *k*-th source code file are related. $\tilde{y}_i^k$ refers to the predicted value of whether the *i*-th bug report and the *k*-th source code file are related. α is the balance factor, and γ is the rate at which the simple sample weight reduction is adjusted.

IV. EXPERIMENTAL SETUP AND RESULTS

First, in this section, the benchmark datasets and evaluation metrics are presented. Then, we analyze our experimental results in three parts. 1) Find the most appropriate correlation matching method. 2) Select the optimal loss function. 3) Compare multiple bug localization models.

A. BENCHMARK DATASETS

To evaluate our experiments, the experimental data we used was from Ye et al. [8], which included four open source Java projects, the Eclipse UI (http://projects.eclipse.org/projects/eclipse.platform.ui), JDT (http://www.eclipse.org/jdt/), SWT (http://www.eclipse.org/swt/) and Tomcat (http://tomcat.apache.org), and these datasets can be publicly obtained. Here, we split these datasets into a training set, a validation set and a testing set. The oldest and newest bug reports are used as the validation set and testing set, respectively. Other bug reports are used as training set. TABLE I gives the number of bugs in each set in detail.

B. EVALUATION METRICS

In this paper, we use Precision, TOP N, MRR and MAP to measure the effectiveness of our experimental results, as follows:

1. Precision: Precision is the ratio of the relevant documen retrieved to all documents retrieved.

2. TOP N: The metric is the number of the top-N (N=1, 5, 10) results obtained by locating buggy code files for bug reports. Each bug report locates at least one relevant buggy code file. The rank position of the relevant code file is different according to different bug localization methods.

3. MRR (Mean Reciprocal Rank): The metric uses the results that bug reports locate relevant buggy code files. The first relevant buggy code file is the most important. The value of MRR is determined by the first relevant buggy code file. The formula for MRR is as follows:

$$MRR = \frac{1}{|S|} \sum_{i=1}^{|S|} \frac{1}{rank_i} \tag{15}$$

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

TABLE I
THE STUDIED PROJECTS

Projects	Time Range	# Total Bug Reports	# Training Set	# Validation Set	# Testing Set
Eclipse UI	2001-10-10 – 2014-01-17	6495	3897	1299	1299
JDT	2001-10-10 – 2014-01-14	6274	3764	1255	1255
SWT	2002-02-19 – 2014-01-17	4151	2491	830	830
Tomcat	2002-07-06 – 2014-01-18	1,056	634	211	211

where $rank_i$ is the result obtained by locating the rank position of the first relevant buggy code file for bug report *i*. $|S|$ refers to the total number of bug reports.

4. MAP (Mean Average Precision): The metric requires the results that all bug reports locate relevant buggy code files. The Average Precision of the *i*-th bug report (AP_i) is as follows:

$$AP_i = \frac{1}{|C_i|}\sum_{j=1}^{|S_i|} Precision(S_i, j) \tag{16}$$

where $|S_i|$ refers to the *i*-th bug report query result list. $|C_i|$ refers to the number of relevant buggy code files for the *i*-th bug report. $Precision(S_i, j)$ refers to the precision of relevant *j*-th buggy code files for the *i*-th bug report query. The formula for MAP is as follows:

$$MAP = \frac{1}{|Q|}\sum_{i=1}^{|Q|} AP_i \tag{17}$$

where $|Q|$ refers to the number of bug reports.

Illustrative Example： Fig. 10 shows an example of evaluation metrics. All queries are A and B, and all documents are the retrieved set. Q_A and Q_B are the A and B query result list. Q_A has two relevant files, and Q_B has three relevant files. At the same time, the precision of each file is calculated. Below, the relevant metrics are calculated according to the following example [38].

Precision： Q_A and Q_B are formed by the A and B query. The precision of Q_A is $P_{Q_A} = 2 \div 6 = 33\%$. The precision of Q_B is $P_{Q_B} = 3 \div 6 = 50\%$.

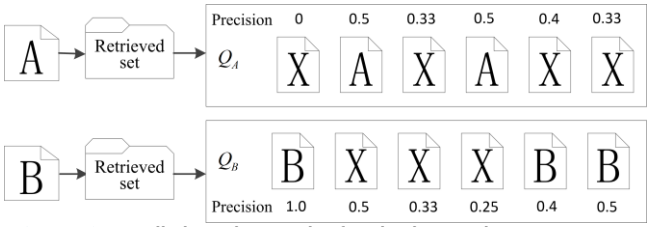

FIGURE 10. A walk-through example of evaluation metrics.

TOP1: The first position of Q_A is a relevant file, and the first position of Q_B is not a relevant file. Therefore, $TOP1 = 1 \div 2 = 50\%$.

TOP5: The first five files of Q_A and Q_B have two relevant files. Therefore, $TOP5 = 2 \div 2 = 100\%$.

TOP10: Since TOP10 includes TOP5. Therefore, $TOP10 = 100\%$.

MRR: The value of MRR is determined by the first relevant file of Q_A and Q_B. The first relevant file is the most important. Therefore, $MRR = (1 \div 2 + 1 \div 1) \div 2 = 0.75$

MAP: The value of MAP is determined by the each relevant file precision for Q_A and Q_B. Therefore, $MAP = ((0.5 + 0.5) \div 2 + (1.0 + 0.4 + 0.5) \div 3) \div 2 = 0.567$.

C. RESEARCH QUESTIONS AND ANALYSIS

RQ1： *How do different correlation matching methods affect bug localization performance?*

Whether it is a bug report based on natural language or a source code file based on a programming language, the correlation matching between them is both surface lexical and semantic correlation matching. In this section, we compare the performance for bug localization under three conditions: surface lexical correlation matching, semantic correlation matching and joint surface lexical and semantic correlation matching.

Fig. 11 shows only the surface lexical correlation matching based on convolutional neural network (SL-CNN). The bug report and source code file form the interaction matrix, which is trained to locate relevant buggy code files for bug reports by a convolutional neural network and focal loss.

Fig. 12 shows only the semantic correlation matching based on a convolutional neural network (S-CNN). We use word2vec to represent the summary of the bug report as a word vector. The description of the bug report is represented as a sentence vector by doc2vec. Each line of the source code file is represented as a word vector by word2vec. Then, their features are extracted by convolutional neural networks. After the Hadamard product is performed between the obtained source code file features and bug report features, the model is trained based on focal loss. The evaluation results show that the bug localization performance of SLS-CNN is better than that of S-CNN and SL-CNN on the validation set.

We use TOP1, TOP5, TOP10, MRR and MAP to evaluate SLS-CNN, S-CNN and SL-CNN. Figs. 13, 14, and 15 show the comparison of four projects (Eclipse UI，JDT，SWT，Tomcat) in TOP1, TOP5, and TOP10. Overall, SLS-CNN is superior to S-CNN, and S-CNN is superior to SL-CNN. In

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

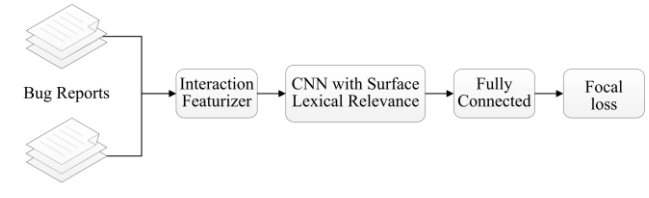

FIGURE 11. **The framework of SL-CNN.**

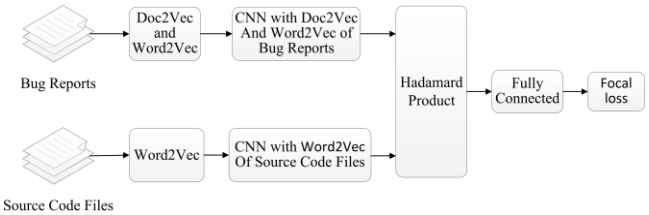

FIGURE 12. **The framework of S-CNN.**

TABLE II
COMPARING SLS-CNN, SL-CNN and S-CNN IN TERMS OF MRR AND MAP

Projects	Methods	MRR	MAP
Eclipse UI	SLS-CNN	**0.55**	**0.46**
	S-CNN	0.45	0.40
	SL-CNN	0.37	0.31
JDT	SLS-CNN	**0.51**	**0.43**
	S-CNN	0.46	0.35
	SL-CNN	0.35	0.29
SWT	SLS-CNN	**0.51**	**0.42**
	S-CNN	0.44	0.37
	SL-CNN	0.37	0.32
Tomcat	SLS-CNN	**0.64**	**0.58**
	S-CNN	0.56	0.49
	SL-CNN	0.46	0.42

these figures, regardless of which method is used, in TOP1, Tomcat's bug localization performance is highest. The TOP1 of SLS-CNN is 56.9%. The TOP1 of S-CNN is 50.0%. The TOP1 for SL-CNN is 34.6%. In TOP5, Tomcat still maintains a good evaluation in each model, but in TOP10, the advantages of Tomcat are not particularly obvious. For example, in the SLS-CNN model, the TOP1 of Tomcat is 56.9%, while the TOP1 of SWT is 40.1%; however, the TOP10 of Tomcat is 84.7%, and the TOP10 of SWT is 79.9%. In TOP10, their evaluation values are not much different. The possible reason is that the three models are more suitable for Tomcat's bug report and software structure.

For TABLE II, we compare three models for four projects using MRR and MAP. SLS-CNN is superior to SL-CNN and S-CNN. SLS-CNN has the highest bug localization performance for Tomcat, and the values of MRR and MAP are 0.64 and 0.58, respectively. This result is consistent with the evaluation of the four software projects with TOP1,

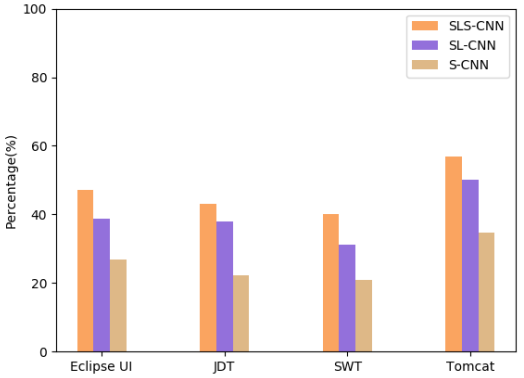

FIGURE 13. **Comparing the three methods in terms of TOP1.**

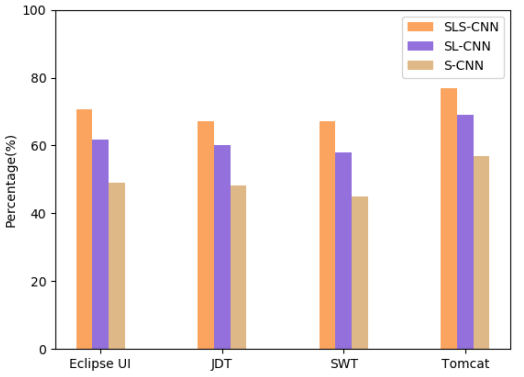

FIGURE 14. **Comparing the three methods in terms of TOP5.**

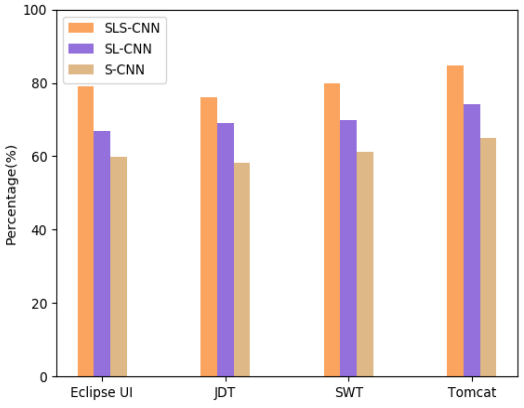

FIGURE 15. **Comparing the three methods in terms of TOP10.**

TOP5 and TOP10. Regardless of whether it is TABLE II or Figs. 13, 14, and 15, the evaluation results show that the bug localization performance for joint surface lexical and semantic correlation matching is higher than considering only surface lexical correlation matching or semantic correlation matching.

RQ2： *Which loss function is better for improving bug localization performance for cross-entropy and focal loss?* For bug localization, the source code file has many files. Each bug report compares all source code files to determine which source code file is buggy and we find only a few

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

TABLE III
COMPARING BUG LOCALIZATION PERFORMANCE for SLS-CNN and CE-CNN

Projects	Methods	TOP1	TOP5	TOP10	MRR	MAP
Eclipse UI	SLS-CNN	**43.2%**	**68.2%**	**75.0%**	**0.52**	**0.42**
	CE-CNN	40.2%	64.1%	70.6%	0.50	0.40
JDT	SLS-CNN	**41.1%**	**64.1%**	**73.4%**	**0.50**	**0.41**
	CE-CNN	37.9%	58.4%	69.1%	0.47	0.38
SWT	SLS-CNN	**36.8%**	**63.7%**	**76.2%**	**0.48**	**0.39**
	CE-CNN	33.2%	59.1%	71.7%	0.45	0.37
Tomcat	SLS-CNN	**53.5%**	**72.5%**	**80.0%**	**0.61**	**0.54**
	CE-CNN	49.1%	67.2%	74.8%	0.55	0.50

buggy source code files related to this bug report, which creates data imbalance. Imbalanced data affects convolutional neural network training, and we use a loss function to solve the impact of data imbalance. Different loss functions have different effects on the bug localization performance. In this paper, the loss function used is focal loss. We compare the bug localization performance of cross-entropy with the bug localization performance of focal loss. The model using cross-entropy as the loss function is named CE-CNN.

As shown in TABLE III, the SLS-CNN and CE-CNN models are evaluated using TOP1, TOP5, TOP10, MRR and MAP. The data used in Table II is the validation set, while the data used in Table III is the testing set. The values of the five evaluation indicators in Table III are smaller than Table II. The possible reason for this phenomenon is that the fit of the validation set is better than the testing set. For the four software projects, the maximum TOP1 of SLS-CNN is 53.5%, and the maximum TOP1 of CE-CNN is 49.1%. For the maximum evaluation of TOP5 and TOP10, the SLS-CNN performance is better than the CE-CNN performance. For SLS-CNN, the MRR values of the four software projects (Eclipse UI, JDT, SWT, and Tomcat) are 0.52, 0.50, 0.48, and 0.61, respectively, and for CE-CNN, the MRR values of the four software projects are 0.50. 0.47, 0.45, and 0.55, respectively. For MAP, The SLS-CNN model is also superior to the CE-CNN model.

It can be seen from the TABLE III that the SLS-CNN model based on focal loss can better solve data imbalance and further improve the bug localization performance.

RQ3：*Is our approach better than other bug localization methods?*

In this section, the SLC-CNN model based on surface lexical and semantic correlation matching of bug reports and source code files is compared with three state-of-the-art models, including HyLoc [10], LR+WE [9] and BugLocator [3]. HyLoc uses deep learning and information retrieval technology to locate bug reports; LR+WE uses word embedding to obtain semantics of bug reports and source code files and then combines other features to enhance the bug localization performance. BugLocator uses a revised vector space model to rank all source code files according to the text similarity, taking into account similar bugs previously fixed

The four methods (SLC-CNN, HyLoc, BugLocator and LR+WE) mainly are evaluated using TOPN, MRR and MAP, and the TOPN is TOP1, TOP2,... TOP10. We compare SLS-CNN, HyLoc, BugLocator and LR+WE on four software projects (Eclipse UI, JDT, SWT, Tomcat). Fig. 16 (A) shows the performance of the four models for Eclipse UI on TOPN. It can be seen that the performance of the SLS-CNN method on TOPN is the best, and the performance of the BugLocator method is the worst. The performances of HyLoc, LR+WE and SLS-CNN are not much different, but The performances of the BugLocator and SLS-CNN method are quite different. For example, the TOP1 of SLS-CNN is 43.2%. The TOP1 of HyLoc is 40.3%. The TOP1 of LR+WE is 39.1%, and the TOP1 of BugLocator is 26.5%. Fig. 16 (B) shows the performance of the four models for JDT on TOPN. It can be seen that the LR+WE is slightly better than the SLS-CNN. The TOP1 of LR+WE is 41.2%, and the TOP1 of SLS-CNN is 41.1%. The TOP5 of LR+WE and SLS-CNN are 65% and 64.1%, respectively. The TOP10 of LR+WE and SLS-CNN are 75% and 73.4%, respectively. The relative performances of the HyLoc and the BugLocator method are poor. Fig. 16 (C) shows the performance of the four models for SWT on TOPN. It can be seen that the performance of the SLS-CNN method is optimal. The performance of the HyLoc method is slightly worse than the LR+WE method on TOP1, but from TOP2 to TOP10, it is better than the LR+WE method. The TOP1 and TOP2 of HyLoc are 31% and 42.8%, respectively, while the TOP1 and TOP2 of LR+WE are 34.2% and 41.1%, respectively. Fig. 16 (D) shows the performance of the four models for Tomcat on TOPN. The performance of the SLS-CNN method is slightly worse than the HyLoc method on TOP2, but on other TOPN, the performance of the SLS-CNN method is better than the HyLoc method. The TOP2 of SLS-CNN is 59.1%, and the TOP2 of HyLoc is 59.6%. From TOP1 to TOP5, the performance gap between HyLoc and SLS-CNN is relatively small, and from TOP5 to TOP10, the performance gap between HyLoc and LR+WE is relatively large. Fig. 17 (A) shows the performance of the four models for four software projects on MRR. It can be seen that the MRR of LR+WE for JDT is the best, and its value is 0.52. The MRR of SLS-CNN for the other three software projects is the best, and the values of Eclipse UI, SWT and Tomcat are 0.52, 0.48 and 0.61 respectively. The MRR of HyLoc and LR+WE for SWT are equal. Fig. 17 (B) shows the performances of four models for four software projects on the MAP. It can be seen that the MAP of LR+WE for JDT is the best, and its value is 042. The MAPs of SLS-

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

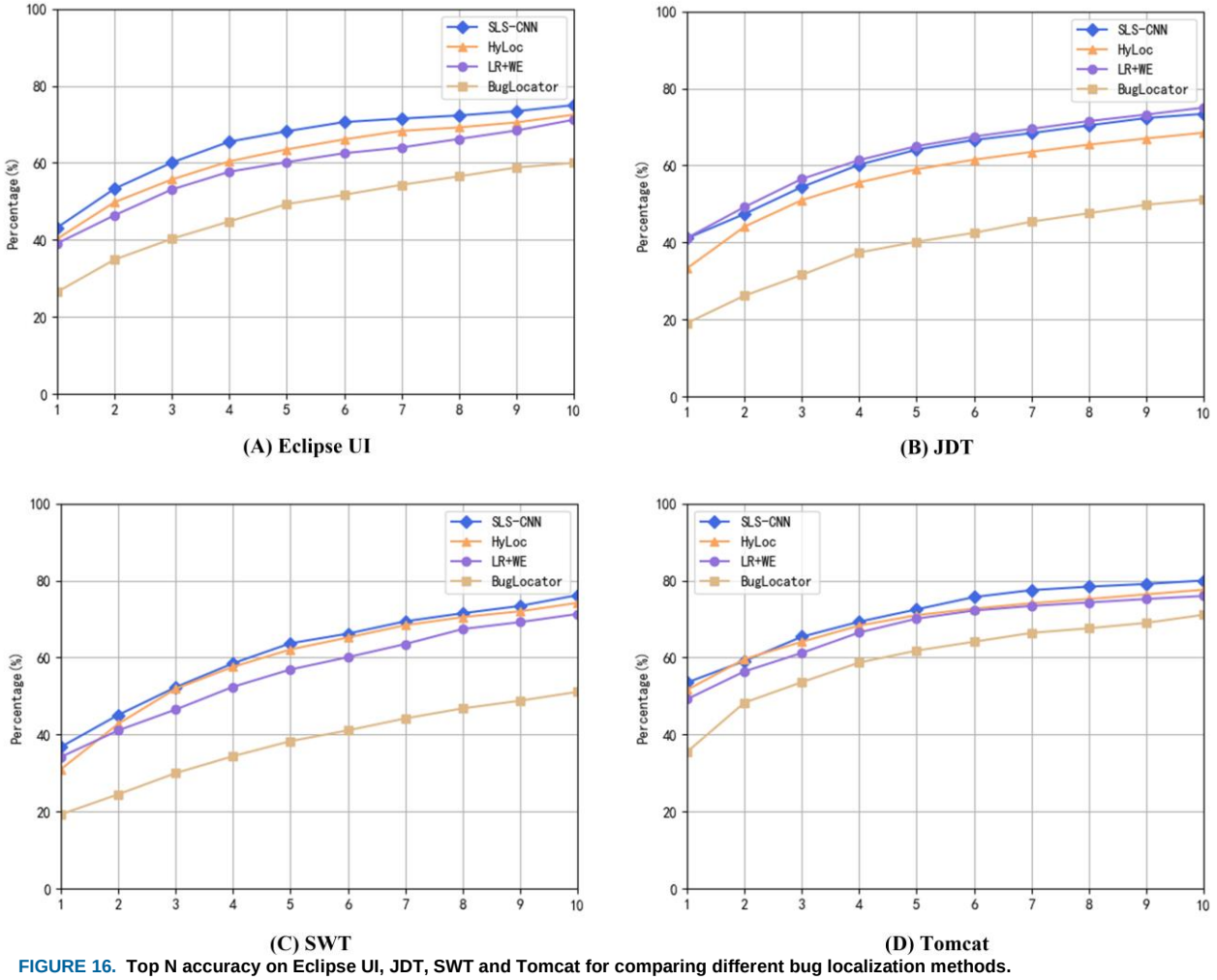

FIGURE 16. Top N accuracy on Eclipse UI, JDT, SWT and Tomcat for comparing different bug localization methods.

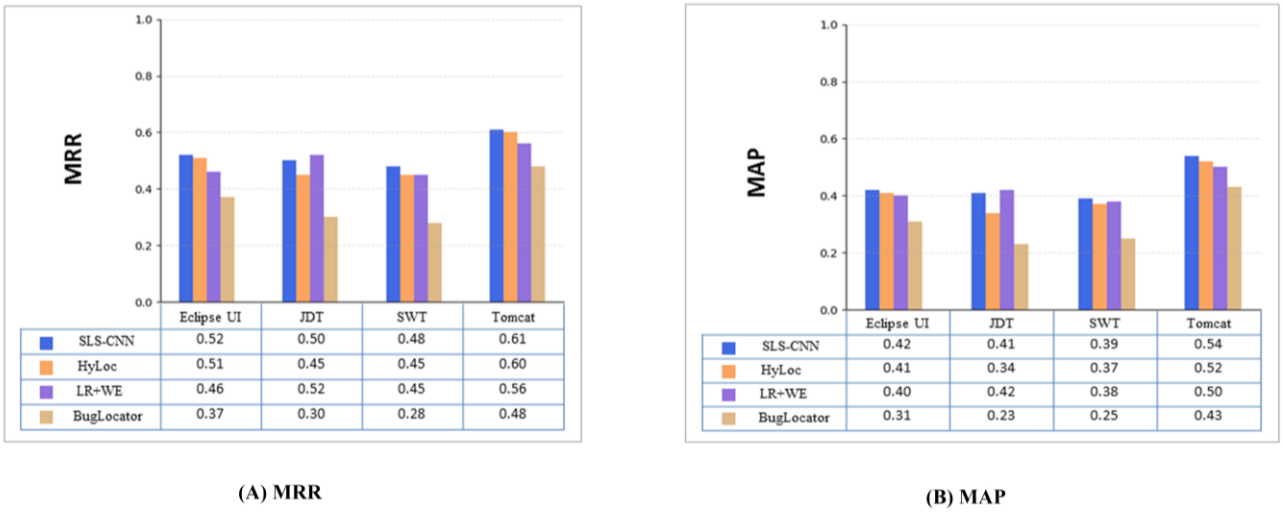

FIGURE 17. Comparing the four methods on four projects in terms of MRR and MAP.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2019.2940557, IEEE Access

CNN for Eclipse UI and Tomcat are the best, and the values are 0.42 and 0.54, respectively.

From the comparison of the four models using TOPN, MRR, and MAP, SLS-CNN obtains the relatively high bug localization performance compared to other methods, but the bug localization performance of LR+WE for JDT is higher than that of SLS-CNN. In general, different bug reports contain different information such as stack traces and program elements. In addition, the source code file structure, bug-fixing recency and frequency, etc. are also different. Since SLS-CNN mainly extracts the features of bug reports and source code files based on convolutional neural networks for bug localization, and LR+WE extracts multiple features including surface lexical similarity of bug reports and source code files, bug-fixing recency and frequency, etc. for bug localization. Therefore, the reason why the bug localization performance of LR+WE for JDT is higher than that of SLS-CNN is the influence of multiple different features.

V. THREATS TO VALIDITY

In our experiments, there are multiple potential threats including internal validity, construct validity and external validity.

Internal Validity. The quality of bug reports and source code files affects the performance of our method. The background knowledge of people who write bug reports and source code files affects their quality. Preprocessing of bug reports and source code files is also important. Good Preprocessing has a positive impact on our method. Different deep network structures also affect the performance of our method. These are very worthy of future studies.

External Validity. In our experiments, we use four projects written in JAVA in the YE article. It is not known whether other projects written in Java or other non-Java languages are suitable for our experiments. In the future, we intend to analyze the impact of projects written in other languages on our approach.

Construct Validity. The datasets used in our experiments are split into a training set, a validation set and a testing set in different proportions. Our approach that we split datasets exists in most machine learning fields. Whether the splitting strategy is optimal or not is uncertain. We leave the effects of different splitting strategies for future studies.

VI. CONCLUSIONS

In the paper, we proposed a novel method named SLS-CNN. This method joint surface lexical and semantic correlation matching based on convolutional neural networks for bug localization. Focal loss was used to solve the data imbalance. We used TOP1, TOP5, TOP10, MRR and MAP to evaluate SLS-CNN, HyLoc, BugLocator and LR+WE on four software projects (Eclipse UI, JDT, SWT, Tomcat). The results show that SLS-CNN obtains the relatively high bug localization performance compared to other methods.

Although our method improves the accuracy of bug localization, it still has some weaknesses. For example, all bug reports are treated as text for the same preprocessing, without considering the impact of different types of bug reports on the bug localization performance. In addition, all source code files are preprocessed based on natural language, without considering the differences between natural language and programming language.

In future, we will extract the features of bug report and source code file in a more efficient way to further improve the bug localization performance. For example, we classify different bug reports to extract more precise features. Since source code files are written in programming language, we rely on structural information, data flow information, dependency information, code length information, etc. to extract more precise features. In addition, better algorithms can be used to improve the accuracy of bug localization. Finally, we will apply our approach to additional types of datasets and other level fine-grained bug localization (such as method level).

REFERENCES

[1] S. K. Lukins, N. A. Kraft, and L. H. Etzkorn, "Bug localization using latent Dirichlet allocation," Information and Software Technology, vol. 52, no. 9, pp. 972-990, Sep. 2010.

[2] G. Gay, S. Haiduc, A. Marcus, and T. Menzies, "On the use of relevance feedback in ir-based concept location," International Conference on Software Maintenance, Edmonton, Canada, 2009, pp. 351–360.

[3] J. Zhou, H. Zhang, and D. Lo, "Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports," International Conference on Software Engineering, Zurich, Switzerland, 2012, pp. 14-24.

[4] R. K. Saha, M. Lease, S. Khurshid, and D. E. Perry, "Improving bug localization using structured information retrieval," International Conference on Automated Software Engineering, Silicon Valley, USA, 2013, pp. 345-355.

[5] K. C. Youm, J. Ahn, and E. Lee, "Improved bug localization based on code change histories and bug reports," Information and Software Technology, vol. 82, pp. 177-192, Feb. 2017.

[6] C. Wong, Y. Xiong, H. Zhang, D. Hao, L. Zhang, and H. Mei, "Boosting Bug-Report-Oriented Fault Localization with Segmentation and Stack-Trace Analysis," IEEE International Conference on Software Maintenance and Evolution, Victoria, Canada, 2014, pp. 181-190.

[7] S. Wang and D. Lo, "AmaLgam+: Composing Rich Information Sources for Accurate Bug Localization," Journal of Software-Evolution And Process, vol. 28, no. 10, pp. 921-942, Oct. 2016.

[8] X. Ye, R. Bunescu, and C. Liu, "Learning to Rank Relevant Files for Bug Reports using Domain Knowledge," ACM SIGSOFT Conference on the Foundations of Software Engineering, Hong Kong, China, 2014, pp. 689-699.

[9] X. Ye, H. Shen, X. Ma, R. Bunescu, and C. Liu, "From Word Embeddings to Document Similarities for Improved Information Retrieval in Software Engineering," International Conference on Software Engineering, Austin, USA, 2016, pp. 404-415.

[10] A. N. Lam, A. T. Nguyen, H. A. Nguyen, and T. N. Nguyen, "Combining Deep Learning with Information Retrieval to Localize Buggy Files for Bug Reports," International Conference on Automated Software Engineering, Lincoln, USA, 2015, pp. 476-481.

[11] X. Huo, M. Li, and Z. Zhou, "Learning Unified Features from Natural and Programming Languages for Locating Buggy Source Code," International Joint Conference on Artificial Intelligence, New York, USA, 2016, pp. 1606-1612.

[12] X. Huo and M. Li, "Enhancing the Unified Features to Locate Buggy Files by Exploiting the Sequential Nature of Source Code," International Joint Conference on Artificial Intelligence, Melbourne, Australia, 2017, pp. 1909-1915.
[13] Y. Xiao, J. Keung, K. E. Bennin, and Q. Mi, "Improving bug localization with word embedding and enhanced convolutional neural networks," Information and Software Technology, vol. 105, pp. 17-29, Jan. 2019.
[14] M. Baroni, G. Dinu, and G. Kruszewski, "Don't count, predict! A systematic comparison of context-counting vs. context-predicting semantic vectors," Meeting of the Association for Computational Linguistics, Baltimore, USA, 2014, pp. 238-247.
[15] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuksa, "Natural Language Processing (Almost) from Scratch," Journal of Machine Learning Research, vol. 12, pp. 2493-2537, Aug. 2011.
[16] D. H. Hubel and T. N. Wiesel, "receptive fields,binocular interaction and functional architecture in the Cat's visual cortex," The Journal of Physiology, vol. 160, no. 1, pp. 106-154, 1962.
[17] K. Fukushima, "Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position," Biological Cybernetics, vol. 36, pp. 193-202, Apr. 1980.
[18] J. Gu, Z. Wang, J. Kuen, and L. Ma, "Recent advances in convolutional neural networks," Pattern Recognition, vol. 77, pp. 354-377, May. 2018.
[19] O. Levy and Y. Goldberg, "Neural Word Embedding as Implicit Matrix Factorization," Advances in Neural Information Processing Systems, Montreal, Canada, 2014, pp. 2177-2185.
[20] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," Advances in Neural Information Processing Systems, Lake Tahoe, USA, 2013, pp. 3111-3119.
[21] T. Mikolov, M. Karafiat, L. Burget, J. H. Cernocky, and S. Khudanpur, "Recurrent neural network based language model," Conference of the International Speech Communication Association, Makuhari, Japan, 2010, pp. 1045-1048.
[22] Y. Bengio, H.Schwenk, J. Senécal, F. Morin, and J. Gauvain, "Neural Probabilistic Language Models," Innovations in Machine Learning, Studies in Fuzziness and Soft Computing, vol. 194, Holmes ,E d. Berlin, GER: Springer, 2006, pp. 137-186.
[23] J. Turian, L. Ratinov, and Y. Bengio, "Word representations: A simple and general method for semi-supervised learning," Meeting of the Association for Computational Linguistics, Uppsala, Sweden, 2010,pp. 384-394.
[24] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," arXiv:1301.3781 [cs.CL],2013. [Online]. Available: https://arxiv.org/abs/ 1301.3781
[25] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global Vectors for Word Representation," Conference on Empirical Methods in Natural Language Processing, Doha, Qatar, 2014, pp.1532-1543.
[26] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," arXiv:1301.3781[cs.CL],2013. [Online]. Available: https://arxiv.org/abs/1301.3781
[27] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," Advances in Neural Information Processing Systems, Lake Tahoe, USA, 2013, pp. 3111-3119.
[28] Q. V. Le and T. Mikolov, "Distributed Representations of Sentences and Documents," International Conference on Machine Learning, Beijing, China, 2014, pp. 1188-1196.
[29] S. Wang and X. Yao, "Using Class Imbalance Learning for Software Defect Prediction," IEEE Transactions on Reliability, vol. 62, no. 2, pp. 434-443, Jun. 2013.
[30] O. F. Arar and K. Ayan, "Software Defect Prediction Using Cost-sensitive Neural Network," Applied Soft Computing, vol. 33, pp. 263-277, Aug. 2015.
[31] V. S. Sheng, B. Gu, W. Fang, and J. Wu, "Cost-sensitive Learning for Defect Escalation," Knowledge Based Systems, vol. 66, pp. 146-155, Aug. 2014.
[32] Z. Xie and M. Li, "Cost-Sensitive margin distribution optimization for software bug localization," Journal of Software, vol. 28, no. 11, pp:3072−3079, 2017.
[33] T. Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal Loss for Dense Object Detection," IEEE International Conference on Computer Vision, Venice, Italy, 2017, pp. 2999-3007.
[34] A. J. Ko, B. A. Myers, and D. H. Chau, "A Linguistic Analysis of How People Describe Software Problems," Visual Languages and Human-Centric Computing, Brighton, England, 2006, pp. 127-134.
[35] G. Capobianco, A. D. Lucia, R. Oliveto, A. Panichella, and S. Panichella, "Improving IR-based traceability recovery via noun-based indexing of software artifacts," Journal Of Software-Evolution And Process, vol. 25, no. 7, pp. 743-762, Jul. 2013.
[36] B. Sisman and A. C. Kak, "Incorporating version histories in Information Retrieval based bug localization," IEEE Working Conference on Mining Software Repositories, Zurich, Switzerland, 2012, pp. 50-59.
[37] Y. Tian and D. Lo, "A comparative study on the effectiveness of part-of-speech tagging techniques on bug reports," IEEE International Conference on Software Analysis, Evolution, and Reengineering, Montreal, Canada, 2015, pp. 570-574.
[38] G. Liu, Y. Lu, K. Shi, J. Chang, and X. Wei, "Mapping Bug Reports to Relevant Source Code Files Based on the Vector Space Model and Word Embedding," IEEE Access, vol. 7, pp. 78870-78881, 2019.